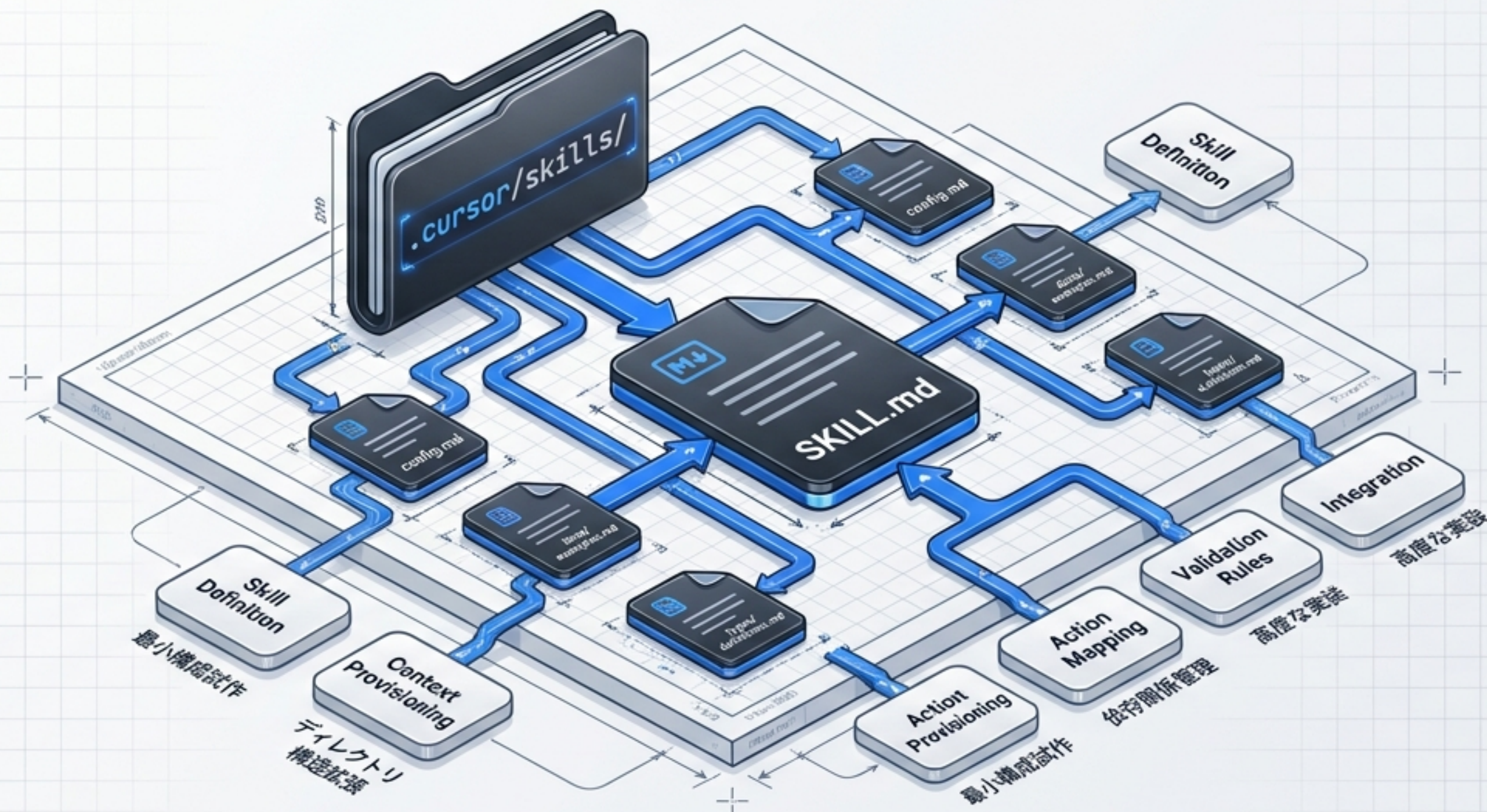




# Cursor Agent Skills 構築実践ガイド

初心者向け：最小構成のお試し作成から実用レベルのディレクトリ拡張までのステップバイステップ完全マニュアル



# 専門知識を動的に読み込ませる「Agent Skills」の構築から拡張までのロードマップ



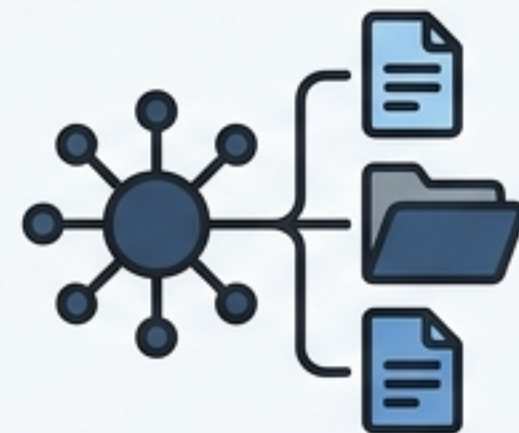
## Phase 1: 準備と基本理解

- Nightlyビルド環境の構築
- 致命的なエラーを防ぐ厳格な命名規則の理解



## Phase 2: 最小構成の稼働

- 1フォルダ・1ファイル (SKILL.md) での立ち上げ
- 発火率を高める記述法と動作確認



## Phase 3: 実用レベルへの拡張

- references/ と scripts/ による役割分割
- 司令塔としての SKILL.md の進化

RulesやCommandsとの決定的な違いは「コンテキストを汚さず、必要な時だけ専門知識を読み込む」こと

## Rules (ルール)



[適用] 常時 (Always On)



[用途] 永続的なコーディング規約、プロジェクト全体の禁止事項



[特徴] 常にコンテキストに入るため、増やしすぎるとAIの判断が鈍る。

## Commands (コマンド)



[適用] 意図的 (Slash Trigger)



[用途] / で呼び出す反復的な特定ワークフロー



[特徴] ユーザーが明示的に呼び出した時のみ実行される。

## Skills (スキル) - 今回の主役



[適用] 動的 & 手動



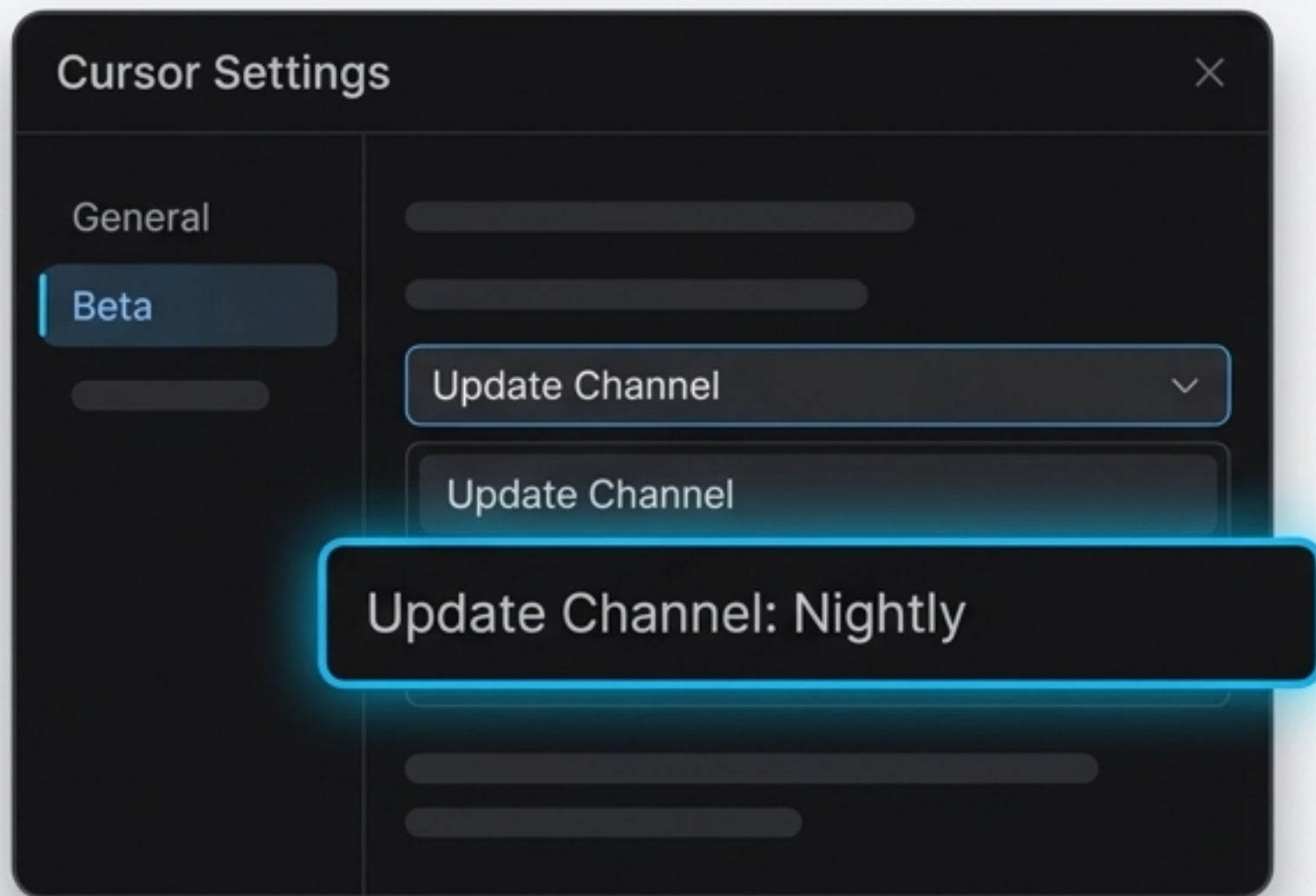
[用途] 特定作業のやり方、専門知識・スクリプトの束



[特徴] Agentが関連性を見て自動適用、または手動で実行。**コンテキストを汚しにくい。**

## Lead Message

**Agent Skillsの利用には「Nightlyリリースチャンネル」への切り替えが必須**



2026年1月現在の公式案内では、  
Agent SkillsはNightlyチャンネル前提。

**1.**

**Settings を開く**

(Cmd/Ctrl + Shift + J)

**2.**

**Beta 項目を選択**

**3.**

**Nightly に切り替えて  
エディタを再起動**

## Lead Message

発火しない最大の原因を回避する、極めて厳格な「命名規則」と「一致」の制約

### Checklist 1: 許可される文字



小文字・数字・ハイフンのみ  
(最大64文字)



大文字、アンダースコア、  
スペース、日本語



先頭・末尾のハイフン、  
連続するハイフン(--)

### Checklist 2: 絶対一致の法則

ディレクトリ名

▼ `.cursor/skills/code-review-lite/`

SKILL.md内のFrontmatter

name: `code-review-lite`



Note: この2つが1文字でも異なると、  
Agentはスキルを認識しません。

## Lead Message

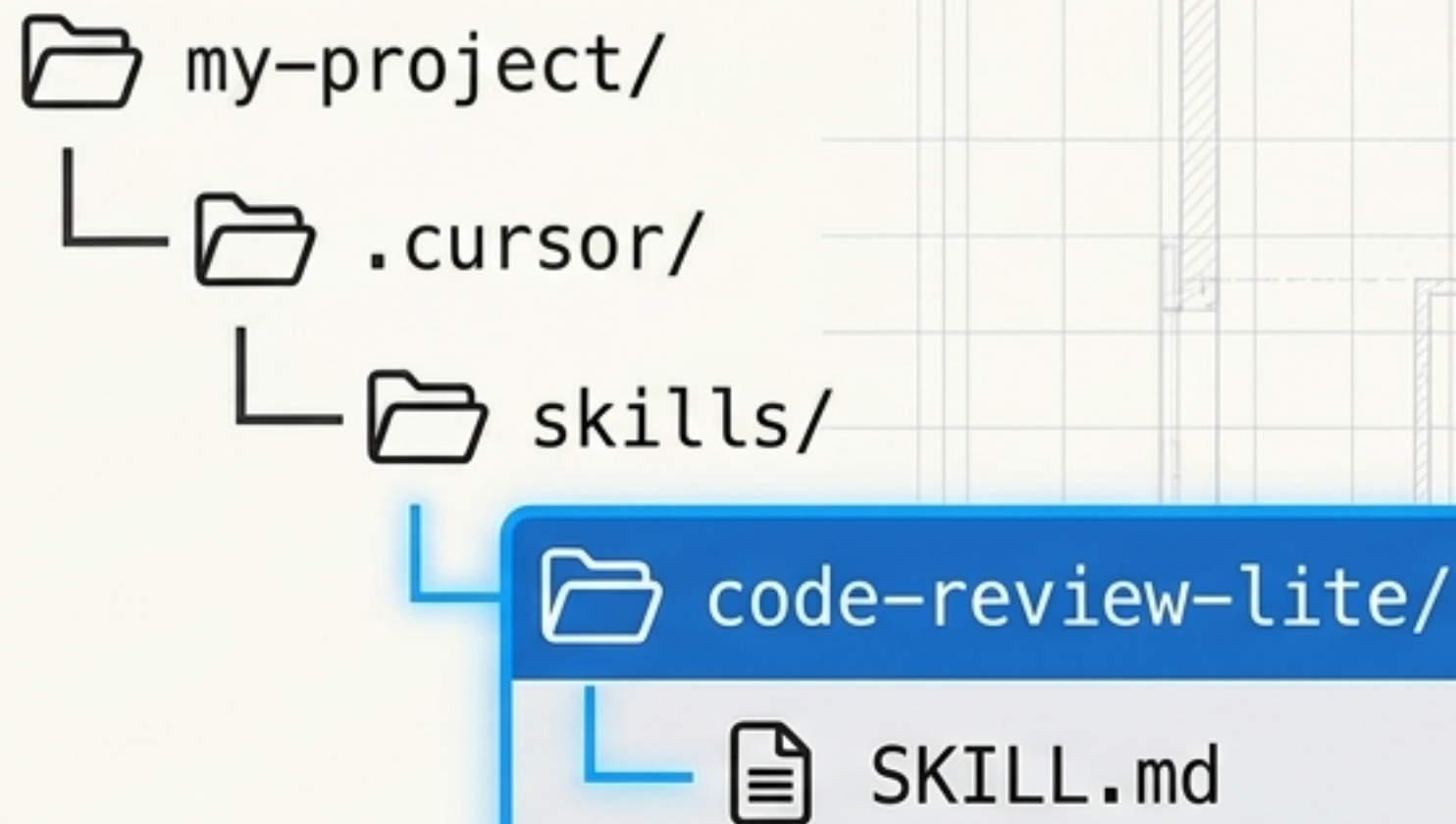
最短で動かす最小構成は、プロジェクト配下にフォルダと SKILL.md を1つ置くだけ

### Action (Left)

```
> mkdir -p .cursor/skills/code-review-lite
```

初心者は「プロジェクト配下 (.cursor/)」に作るのが基本。Git管理しやすく、壊してもすぐリセット可能。(全プロジェクト共通なら ~/.cursor/skills/)

### Directory Tree (Right)



## 発火精度は description に依存する。SKILL.md の「Frontmatter」と「Body」の解剖学

フォルダ名と完全一致必須の識別子

```
SKILL.md

---
name: code-review-lite
description: ユーザーが「PRレビュー」「差分レビュー」「バグの指摘」を依頼した時にこのスキルを使用する。
---
```

### # コードレビュー手順

1. 変更されたファイルの差分を確認する。
2. メモリリークやセキュリティの脆弱性がないかチェックする。
3. 改善案をマークダウン形式で出力する。

✗ 「何をするか」ではなく、  
✓ 「どんな依頼の時に使うか」を明記

本文は5~10ステップ程度に収め、  
「どう振る舞うか」を日本語で簡潔に指示

# スキルの認識を確認し、「自動発火」または「手動発火」でAgentに指示を出す



## Step 1: 認識の確認

### Settings > Rules

(Cmd/Ctrl + Shift + J)を開き、作成したスキルが表示されているか確認。



## Step 2: 自動適用

Agent Chatにて自然文で依頼。「この差分レビューして」

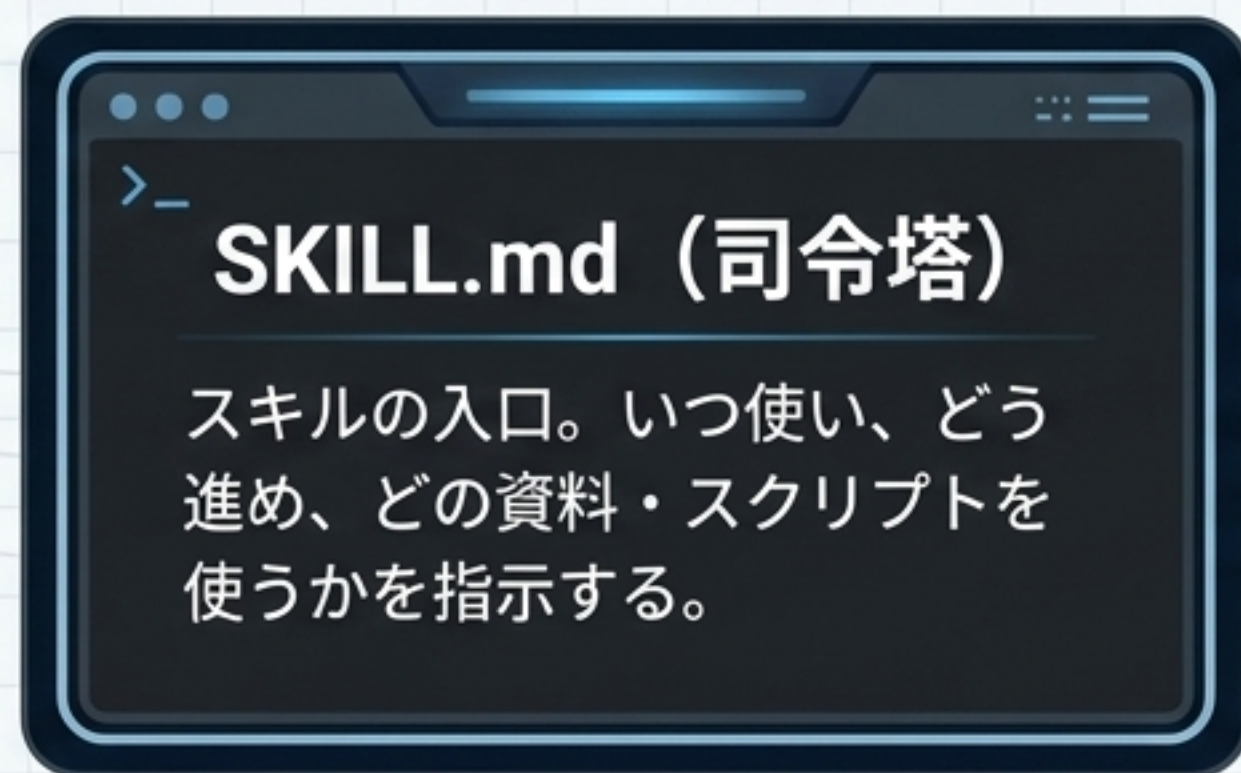
Agentが文脈と description を照合し、自動でスキルを読み込んで実行。



## Step 3: 手動適用

Agent Chatにて  
`/code-review-lite` 差分をレビューしてくださいと入力。  
スラッシュコマンドによる確実な呼び出し。

## ■ 実用レベルへの拡張：SKILL.mdを「司令塔」とし、参照資料と定型処理を分離する



### references/ (参照・詳細ルール)

毎回SKILL.mdに書くと長すぎる詳細な判断基準（チェックリスト、規約、NG/OK例など）。



### scripts/ (自動化処理)

人が毎回手で打つのが面倒な定型コマンド（差分収集、ログ保存など）のスクリプト群。

# 判断基準や長文のルールは references/ に分離し、保守性を高める

code-review-lite/

SKILL.md

references/

review-checklist.md

test-guidelines.md

review-checklist.md

- 変数名は意図が伝わるか
- N+1問題が発生していないか

test-guidelines.md

- 境界値テストが含まれているか
- モックの使い方は適切か

① Note: これらは「毎回読む可能性はあるが、直書きすると長すぎるもの」を逃がす場所。

差分取得やログ保存などの定型コマンドは `scripts/` にまとめ、Agentに実行させる

`scripts/collect_diff.sh`

```
#!/bin/bash
```

```
git diff main > .review_diff.txt
```

```
echo "Diff saved to .review_diff.txt"
```



Agentに毎回 `git diff main` を手動で指示するのではなく、このスクリプトを呼び出させることで手順を完全自動化。

## CRITICAL WARNING

スクリプトには必ず実行権限 (`chmod +x scripts/collect_diff.sh`) を付与のすること。権限がないとAgentは実行できません。

## LEAD MESSAGE

分離したファイルは、SKILL.md内に「相対パス」で参照先と実行タイミングを明記して結びつける

### SKILL.md

1. 変更ファイルの特定:  
`scripts/collect_diff.sh` を実行し、  
差分を取得すること。
2. レビューの実施:  
`references/review-checklist.md`  
の観点に従って差分をチェックすること。



`collect_diff.sh`

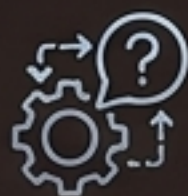
実行タイミング:  
変更発生時、自動実行



`review-checklist.md`

参照タイミング:  
レビュー時、自動参照

### KEY INSIGHT



ファイルを置くだけではAgentは使いません。  
「いつ読むか」「いつ実行するか」を司令塔に明記すること（skill rootからの相対パスを使用）。

# 安定運用のための「日本語化」ベストプラクティス：識別子は英字、指示は日本語



## Rule Matrix: 識別子と指示言語の分離

|                                      |                                       |                                               |
|--------------------------------------|---------------------------------------|-----------------------------------------------|
| name と<br>ディレクトリ名                    | [英数字・ハイフンのみ]<br>(例: code-review-lite) | -> 識別子として安定稼働させるため。                           |
| description                          | [日本語OK]                               | -> Agentがユーザーの自然文（日本語）と<br>関連性をマッチングしやすくするため。 |
| 本文 (Body)                            | [日本語OK]                               | -> 複雑な手順やニュアンスを正確に伝える<br>ため。                  |
| ファイル名<br>(references/<br>scripts/配下) | [英字推奨]                                | -> 相対パス参照や環境差異によるエラー<br>を防ぐため。                |

## 失敗しないためのトラブルシューティング：よくある躓きとその解決策

| 現象 / Problem                                                                                                   | 対策 / Solution                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  自然文で話しかけても発火しない。             |  description を「これはコードをレビューします(悪い例)」から「ユーザーがPRレビューやバグ確認を依頼した時に使う(良い例)」に書き直す。 |
|  / (スラッシュ) パレットにスキル名が出てこない。 |  Nightlyビルド固有の表示揺れの可能性あり。設定のBetaチャンネルと、フォルダ名/nameの一致を再確認する。               |
|  参照ファイルがうまく読み込まれない。         |  SKILL.md に全部詰め込んでいないか？<br>あるいは、参照が深すぎないか？<br>相対パス参照はシンプルに保つ。             |

まずは「毎回似た手順を踏む作業」の自動化から。おすすめの初期スキル3選



### コードレビューSkill

今あるリポジトリですぐに試せる。差分レビュー、PRレビュー、テスト漏れ確認など、Skillsの効果が最も見えやすい。



### ブログ執筆Skill

ZennやQiita向け。記事のFrontmatterテンプレート適用、文体ルールの統一、技術記事の注意点の自動チェック。



### プロジェクト固有チェックSkill

「mainとの差分を見る」「社内規約（APIハンドラに認証チェック必須など）に準拠しているか見る」など、実務直結のカスタマイズ。

「まずは SKILL.md の1ファイル構成から始め、安定したら references/ と scripts/ を足して実用レベルへ育てましょう。」